

Simulation Without a Black Box

Outline

Why Deep RL?

DQN

Experience Replay

Target Network

PPO as Alternative

Hyperparameter Tuning and Reward Normalization

Policy Comparison

Key Takeaways

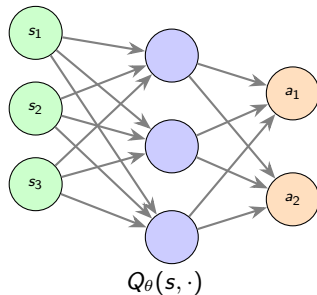
Why Deep RL: Large and Continuous State Spaces

Tabular Q-learning fails when:

- State space is too large to enumerate (multi-product, multi-server)
- Observations include continuous values (utilisation, inter-arrival time)
- Features interact in complex ways that simple bins miss

Deep RL solution:

Replace the table $Q[s, a]$ with a neural network $Q_\theta(s, a)$ that *generalises* across states.



DQN: Deep Q-Network

Key Idea (Mnih et al., 2015)

Approximate $Q^*(s, a)$ with a neural network $Q_\theta(s, a)$.

Loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y - Q_\theta(s, a))^2 \right]$$

where the **TD target** is:

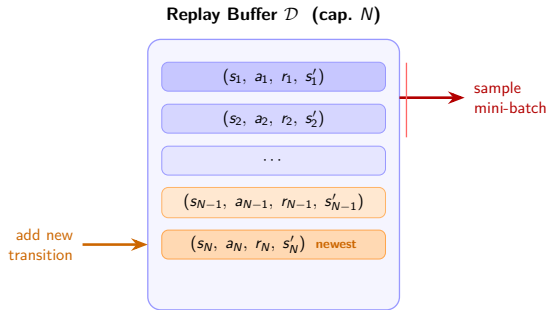
$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$$

θ^- : parameters of a *target network* (held fixed during gradient steps).

Two stabilization tricks:

1. **Experience Replay**: sample random mini-batches from a replay buffer $\mathcal{D}$ rather than using consecutive (correlated) transitions.
2. **Target Network**: use a periodically-copied frozen copy θ^- to compute targets, preventing moving-target instability.

Experience Replay Buffer



Why replay helps:

- Breaks temporal correlation between consecutive observations
- Each transition can be replayed many times ($\Rightarrow$ data efficiency)
- Uniform sampling or **prioritized** replay (PER) — sample high-TD-error transitions more often

Memory Budget

Buffer size: 10 000–1 000 000 transitions. For large observation vectors, buffer RAM can dominate total memory usage.

Target Network for Training Stability

Without a target network:

- TD target y depends on θ (same parameters being updated)
- $\Rightarrow$ target moves with every gradient step
- $\Rightarrow$ oscillations, divergence

With a target network θ^- :

- θ^- is a copy of θ updated every C steps (or via exponential moving average)
- Targets are computed from the frozen θ^-
- $\Rightarrow$ stable, slowly-moving target surface

```
# stable-baselines3 DQN
from stable_baselines3 import DQN
model = DQN(
    policy="MlpPolicy",
    env=env,
    learning_rate=1e-4,
    buffer_size=50_000,
    learning_starts=1_000,
    batch_size=64,
    tau=1.0, # hard update
    target_update_interval=500,
    gamma=0.99,
    verbose=1,
)
model.learn(total_timesteps=100_000)
```

PPO: Proximal Policy Optimization

PPO (Schulman et al., 2017)

On-policy algorithm that directly optimises a *policy* $\pi_{\theta}(a|s)$ using a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right]$$

where $r_t = \pi_{\theta} / \pi_{\theta_{\text{old}}}$ and $\hat{A}_t$ is the advantage estimate.

DQN vs. PPO comparison:

	DQN	PPO
Policy type	Determin.	Stochastic
Data reuse	Off-policy	On-policy
Action space	Discrete	Disc./cont.
Sample eff.	Higher	Lower
Stability	Moderate	High

Reward Normalization and Hyperparameter Tuning

Reward normalization:

Unnormalized rewards (e.g., wait in seconds) can be very large. Standardize:

$$\tilde{r}_t = \frac{r_t - \mu_r}{\sigma_r + \epsilon}$$

Use a running estimate of μ_r and σ_r during training. **Key hyperparameters to tune:**

- Learning rate α : start at 3×10^{-4}
- Discount γ : 0.99 for long-horizon problems
- Replay buffer size (DQN): 10^4 – 10^6
- Network size: 2–3 hidden layers of 64–256 units

Tuning Trap

Never tune hyperparameters on the test set — use a separate validation environment with different seeds. Report hyperparameters in the paper.

Quick Diagnostic

Plot episode return vs. timestep.

- Flat line $\Rightarrow$ not learning: check reward / LR
- Rising then falling $\Rightarrow$ overfit: reduce LR or add entropy
- Oscillating $\Rightarrow$ instability: reduce LR or add target network delay

Comparing RL Policy to Baselines

Evaluation baselines (in increasing sophistication):

1. **Myopic policy:** always choose lowest-queue server (greedy now)
2. **Expert heuristic:** e.g., open server if $q \geq 5$ (practitioner rule)
3. **SimOpt solution:** best static staffing from L37 (optimised but non-adaptive)
4. **RL policy:** adaptive, learned from experience

A learned policy that does not beat the expert heuristic is not ready for deployment.

Policy	$\bar{W}_q$	Staff cost	All
Myopic	4.2	High	
Expert ($q \geq 5$)	3.1	Medium	
SimOpt (static)	2.8	Medium	
DQN (adaptive)	2.1	Low	

figures: 500 evaluation episodes, same seeds.

Key Takeaways

L42 Summary

1. Deep RL replaces the Q-table with a neural network $Q_{\theta}(s, a)$, enabling generalisation across large or continuous state spaces.
2. DQN uses two stabilisation tricks: experience replay (break correlation) and a target network (prevent moving targets).
3. PPO is an on-policy alternative with better stability and support for continuous action spaces.
4. Normalize rewards; tune learning rate and network size carefully; diagnose training from the learning curve.
5. A trained policy must be compared to myopic, expert, and SimOpt baselines under common evaluation seeds before any deployment claim.

Module M12 complete. See `notebooks/L42_deep_rl_training.ipynb` for hands-on experiments.